



## LeetCode 682: Baseball Game

### 1. Problem Title & Link

**Title:** LeetCode 682: Baseball Game

**Link:** <https://leetcode.com/problems/baseball-game/>

### 2. Problem Statement (Short Summary)

You are keeping score for a baseball game.

You are given an array of operations ops.

Each operation can be:

| Operation | Meaning                           |
|-----------|-----------------------------------|
| Integer x | Record score x                    |
| "+"       | Record sum of previous two scores |
| "D"       | Record double of previous score   |
| "C"       | Remove previous score             |

Return the total score after processing all operations.

### Example

Input:

ops = ["5", "2", "C", "D", "+"]

Output:

30

Explanation:

"5" → [5]

"2" → [5,2]

"C" → [5]

"D" → [5,10]

"+" → [5,10,15]

Total:

5 + 10 + 15 = 30

### 3. Examples (Input → Output)

#### Example 1



Input:

```
["5","2","C","D","+"]
```

Output:

30

### Example 2

Input:

```
["5","-2","4","C","D","9","+","+"]
```

Output:

27

### Example 3

Input:

```
["1"]
```

Output:

1

## 4. Constraints

$1 \leq \text{ops.length} \leq 1000$

`ops[i]` is:

Integer

"C"

"D"

"+"

Problem guarantees operations are valid.

## 5. Core Concept (Pattern / Topic)

### Stack

Secondary Topics:

- Simulation
- Array Processing

## 6. Thought Process (Step-by-Step Explanation)

### Understanding the Operations

We need to remember previous scores.

Example:



5

2

+

For +:

Need previous two scores

5 and 2

Result:

$5 + 2 = 7$

For D:

Double previous score

Example:

5

Becomes:

10

For C:

Remove previous score

Example:

[5,2]

After C:

[5]

### Why Stack?

We always need access to:

Most recent score

Previous score

This is exactly what a Stack provides.

### Example

Input:

["5","2","C","D","+"]

Process:

5



Stack:

[5]

Process:

2

Stack:

[5,2]

Process:

C

Remove top.

Stack:

[5]

Process:

D

Double last score:

10

Stack:

[5,10]

Process:

+

Previous two:

$5 + 10 = 15$

Stack:

[5,10,15]

Total:

30

## 7. Visual / Intuition Diagram (ASCII Diagram)

Input:

["5", "2", "C", "D", "+"]

5

Stack: [5]

2

Stack: [5,2]



```
C
Stack: [5]

D
Stack: [5,10]

+
Stack: [5,10,15]
Answer:
30
```

## 8. Pseudocode (Language Independent)

```
create stack

for operation in ops
    if integer
        push integer
    else if "C"
        pop
    else if "D"
        push 2 * top
    else if "+"
        push top + secondTop

return sum(stack)
```

## 9. Code Implementation

### Python

```
class Solution:
    def calPoints(self, ops):

        stack = []

        for op in ops:

            if op == "C":

                stack.pop()
```



```
        elif op == "D":

            stack.append(2 * stack[-1])

        elif op == "+":

            stack.append(
                stack[-1] + stack[-2]
            )

        else:

            stack.append(int(op))

    return sum(stack)
```

## Java

```
import java.util.*;

class Solution {

    public int calPoints(String[] ops) {

        Stack<Integer> stack =
            new Stack<>();

        for(String op : ops) {

            if(op.equals("C")) {

                stack.pop();

            }

            else if(op.equals("D")) {

                stack.push(
                    2 * stack.peek()
                );

            }

            else if(op.equals("+")) {

                int first = stack.pop();
                int second = stack.peek();

                stack.push(first);

                stack.push(first + second);

            }

        }

    }

}
```



```
        }

        else {

            stack.push(
                Integer.parseInt(op)
            );
        }
    }

    int sum = 0;

    for(int score : stack)
        sum += score;

    return sum;
}
}
```

## 10. Time & Space Complexity

### Time Complexity

$O(n)$

Each operation processed once.

### Space Complexity

$O(n)$

Stack stores scores.

## 11. Common Mistakes / Edge Cases

### Mistake 1

Using variables instead of stack.

Need access to:

Last score

Second-last score

### Mistake 2

Removing score incorrectly for:

C

Must completely remove previous score.



### Mistake 3

For +

Using:

Current total

instead of:

Previous two scores

### Edge Cases

#### Single Score

Input:

["5"]

Output:

5

#### Negative Score

Input:

["-5"]

Output:

-5

#### Multiple Removes

Problem guarantees validity.

No need to handle invalid operations.

### 12. Detailed Dry Run

Input:

["5","2","C","D","+"]

| Operation | Stack     |
|-----------|-----------|
| 5         | [5]       |
| 2         | [5,2]     |
| C         | [5]       |
| D         | [5,10]    |
| +         | [5,10,15] |

Sum:

$5 + 10 + 15 = 30$

Return:



30

### 13. Common Use Cases (Real-Life / Interview)

#### Undo Operations

C

behaves like undo.

#### History Tracking

Store previous states.

#### Financial Transactions

Rolling updates using recent records.

#### Interview Purpose

Tests:

Stack

Simulation

History Management

### 14. Common Traps (Important!)

#### Trap 1

Not recognizing stack usage.

#### Trap 2

Modifying total directly.

Need score history.

#### Trap 3

Confusing:

+

with total sum.

It means:

Last two scores sum

### 15. Builds To (Related LeetCode Problems)

LC 682 - Baseball Game





LC 20 - Valid Parentheses



LC 1047 - Remove All Adjacent Duplicates



LC 844 - Backspace String Compare



LC 155 - Min Stack

## 16. Alternate Approaches + Comparison

| Approach     | Time   | Space  | Interview Rating |
|--------------|--------|--------|------------------|
| List History | $O(n)$ | $O(n)$ | Good             |
| Stack        | $O(n)$ | $O(n)$ | Best             |

## 17. Why This Solution Works (Short Intuition)

Every operation depends on the most recent scores.

A stack naturally stores score history and allows quick access to the latest entries, making all operations efficient.

## 18. Variations / Follow-Up Questions

### Follow-Up 1

Support:

Triple previous score operation.

### Follow-Up 2

Add redo functionality.

### Follow-Up 3

Return score history instead of total.

### Follow-Up 4

Support deleting last K scores.

## Interview Takeaway

Whenever you hear:

Undo



History

Recent Operation

Last K Values

think:

STACK

A stack is often the simplest and most natural way to manage reversible operations and recent-state dependencies.